**Infraestrutura de software**
**Guia de Relatorios**

**Atenção**

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

# 1. Objetivo (3–5 linhas)

O objetivo desta atividade foi implementar, em C, a sincronização entre as threads dos passageiros e dos vagões de metrô. O programa deve permitir que passageiros embarquem quando houver assentos disponíveis, impedindo que o vagão parta antes da confirmação dos embarques. A entrega inclui as funções solicitadas, os arquivos de compilação e os registros dos testes realizados.

# 2. Checklist de Requisitos

| Requisito do enunciado | Status | Como testar / evidência |
|---|---|---|
| Implementação em C | OK | Arquivos sync_car.c e sync_car.h |
| Inicialização da estação | OK | Função station_init |
| Sincronização da chegada do vagão | OK | station_load_car e runner, log linhas 19–24 |
| Espera dos passageiros | OK | station_wait_for_car e runner |
| Confirmação dos embarques | OK | station_on_board e runner |
| Funções pthread exigidas | OK | Inspeção do código-fonte |
| Apenas um mutex por estação | OK | Struct station, no arquivo sync_car.h |
| Não utilizar semáforos | OK | Inspeção do código |
| Permitir embarques simultâneos | OK nos cenários testados | Reserva de assentos antes da confirmação; runner |
| Não utilizar busy-wait na implementação | OK | Uso de pthread_cond_wait |
| Compilação por Makefile | OK | Log linhas 8–17 e 201–212 |
| Executável sync_car | OK | Compilação registrada no log, linhas 10–17 |
| Testes de entradas inválidas | Não comprovado | Comando iniciado na linha 229, sem resultado registrado |
| Testes de casos limites | Parcial | Runner verifica carro sem assentos e ausência de passageiros esperando |

# 3. Como Reproduzir

**Ambiente utilizado:** Windows com WSL Ubuntu 24.04, GCC 13.3.0 e GNU Make 4.3.

**Compilar:**

make clean

make

**Executar:**

./sync_car

**Executar pelo Makefile:**

make test

**Repetir o teste 100 vezes:**

for i in $(seq 1 100); do timeout 20 ./sync_car || break; done

**Limpar os arquivos compilados:**

make clean

# 4. Arquitetura (resumida)

**Arquivos e responsabilidades:**

1. sync_car.h → Declara a struct station e as funções de sincronização.
2. sync_car.c → Implementa o controle de embarque e a comunicação entre as threads.
3. sync_car-runner.c → Arquivo de testes fornecido pelo professor, utilizado para verificar o funcionamento.
4. Makefile → Compila, executa os testes e limpa os arquivos gerados.
5. README.md → Explica a organização e como executar o projeto.
6. evidencias.log → Registra as sessões de teste realizadas no terminal.

**Decisões técnicas:**

1. Separar .h e .c → Queria evitar arquivos muito longos, como aconteceu em implementações anteriores → Ficou mais fácil organizar as declarações e as funções.
2. Utilizar um único mutex → Proteger os contadores compartilhados entre as threads → As alterações nos contadores acontecem dentro de regiões críticas.
3. Utilizar duas variáveis de condição → Passageiros precisam esperar pelo carro, e o carro precisa esperar pelas confirmações → As threads podem ficar bloqueadas sem consumir processamento em verificações contínuas.

# 5. Estratégias e Diário de Desenvolvimento

## 5.1 Estratégias da semana

| Estratégia | Nome curto | Contexto | Motivo da troca (se houve) |
|---|---|---|---|
| S1 | Contadores, mutex e condições | Organizei a implementação em quatro funções, utilizando três contadores, um mutex e duas variáveis de condição. | Não houve troca de estratégia principal documentada. |

## 5.2 Diário de Tentativas

| # | Estrat. | O que tentei | Resultado | Hipótese /Causa (se falhou) | Quando | Evidência |
|---|---|---|---|---|---|---|
| 1 | S1 | Executei make clean e make. | Compilação concluída, com um aviso no runner. | - | 02/11, 19h03 | Log, linhas 8–17 |
| 2 | S1 | Executei o programa com timeout 10 para observar a saída. | Terminou com Looks good!. | - | 02/11, após 19h03 | Log, linhas 19–24 |
| 3 | S1 | Recompilei e realizei outras execuções do programa. | Novas execuções terminaram com Looks good!. | - | 02/11, 19h08–19h11 | Log, linhas 51–104 |
| 4 | S1 | Executei o programa e verifiquei seu código de saída. | Looks good! e código de saída 0. | - | 04/11, sessão das 17h10 | Log, linhas 128–138 |
| 5 | S1 | Recompilei e repeti a execução 100 vezes. | 100 sucessos e 0 falhas. | - | 04/11, sessão das 17h20 | Log, linhas 166–185 |
| 6 | S1 | Fiz outra compilação e repeti os testes. | Compilação concluída, código de saída 0 e 100 sucessos. | - | 04/11, sessão das 18h14 | Log, linhas 201–228 |
| 7 | S1 | Iniciei um comando para compilar um teste chamado teste_inv. | Não há resultado registrado. | O arquivo termina antes da conclusão do teste. | 04/11, última sessão | Log, linha 229 |

# 6. Evidências

## 6.1. Log automático

O arquivo evidencias.log registra os comandos de compilação e os testes realizados nos dias 02/11 e 04/11.

02/11/2026 :Compilação e primeiros testes
Executei make clean e make, gerando o executável sync_car. A compilação apresentou apenas um aviso no runner do professor. Nas execuções seguintes, o programa terminou com Looks good!, e as quantidades de passageiros embarcados coincidiram com as esperadas. Os resultados aparecem nas linhas 8–24 e 51–104 do log.

04/11/2026 :Validação dos testes
Recompilei o programa, confirmei o código de saída zero e realizei duas sequências de 100 execuções, ambas com sucessos=100 falhas=0. Os registros aparecem nas linhas 128–138, 166–185 e 201–228 do log.
Os resultados foram positivos nos cenários executados, mas não garantem a ausência de problemas em todas as situações de concorrência.

## 6.2 Prints

Figura 1: Compilação do projeto utilizando make clean e make. O executável foi gerado, apresentando apenas o aviso unused parameter 'foo', no arquivo sync_car-runner.c fornecido pelo professor.

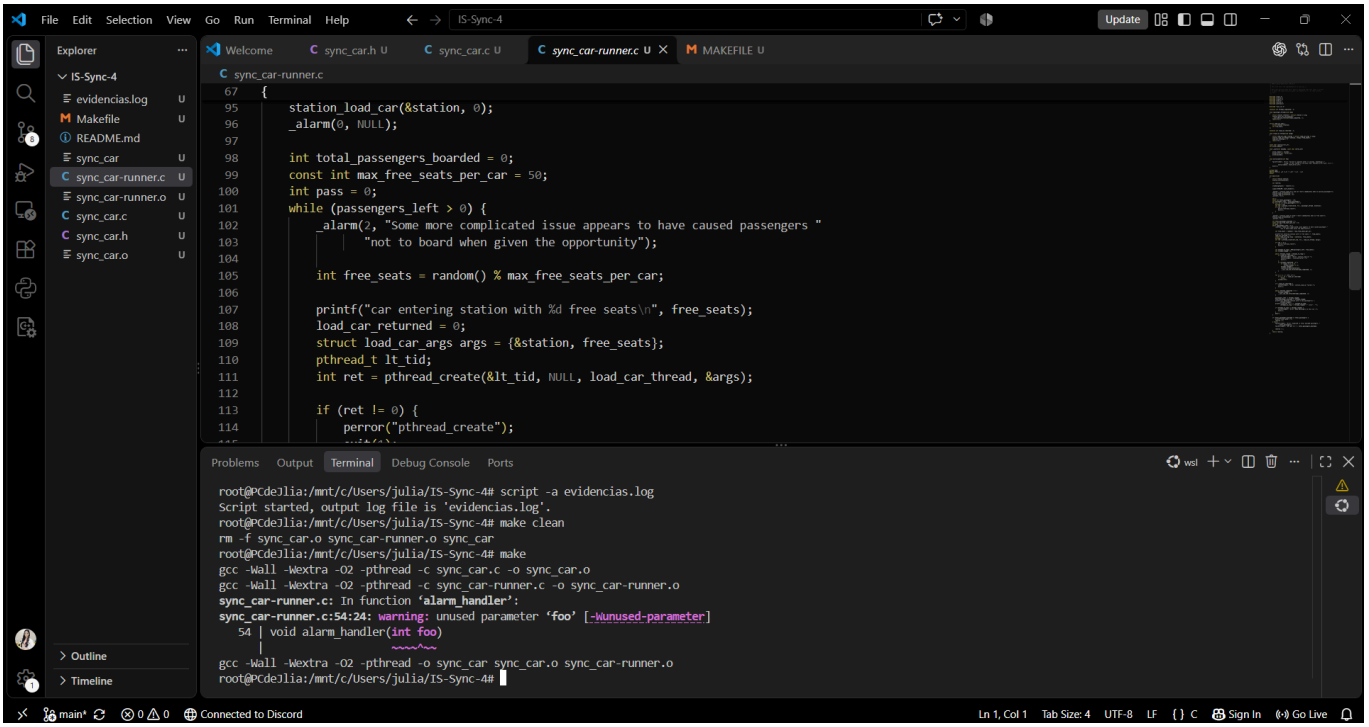
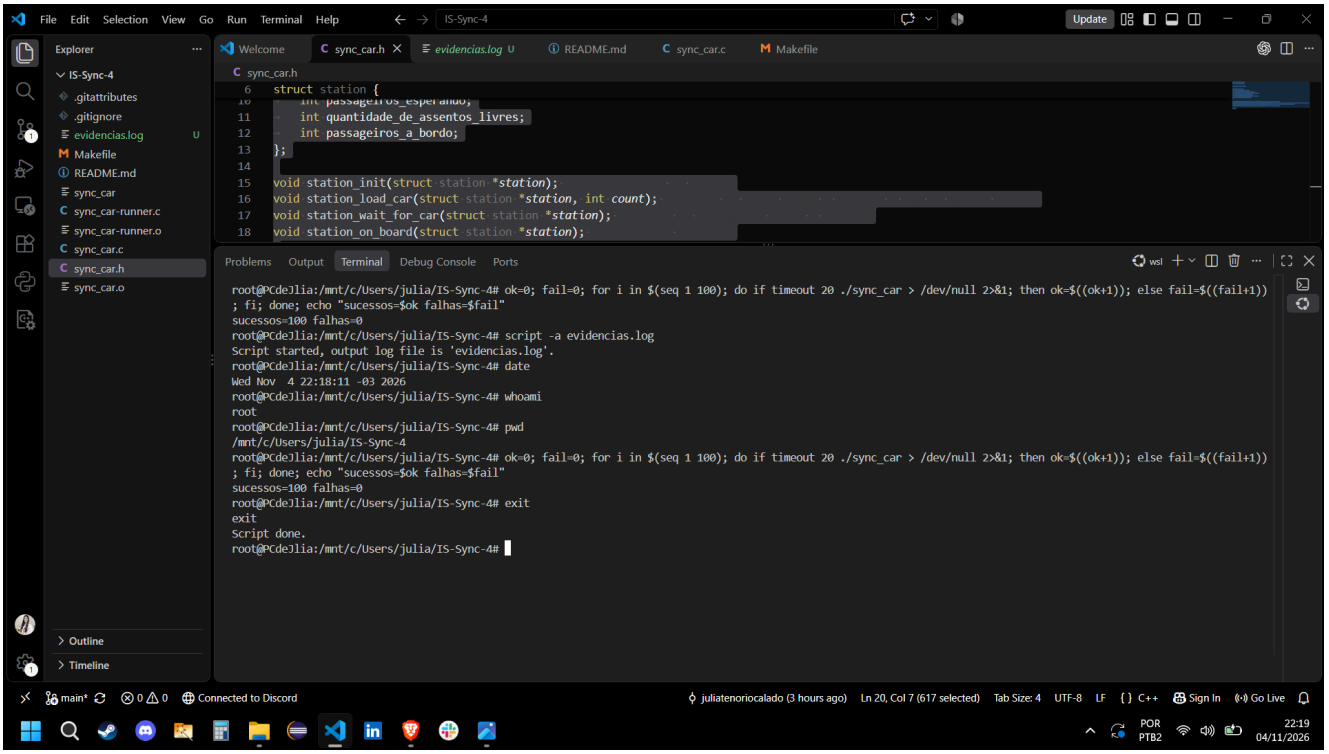



Figura 2: Validação final do programa após a repetição do runner 100 vezes. O terminal apresentou sucessos=100 falhas=0, indicando que todas as execuções retornaram sucesso dentro do limite estabelecido.




Não foi identificado um erro principal da implementação com resultado de falha documentado no log apresentado. Por isso, o primeiro print registra o aviso de compilação, sem classificá-lo como bug do código desenvolvido.

# 7. Uso de IA

**Onde usei IA (2–3 linhas):** Durante a atividade, utilizei IA como apoio para revisar a implementação que já havia desenvolvido e verificar se ela atendia aos requisitos do enunciado. Também pedi ajuda para avaliar a suficiência dos testes e organizar o relatório a partir dos registros do terminal e das minhas anotações de desenvolvimento.

**Prompts principais:**

- Revisão das funções implementadas para verificar se atendiam aos requisitos do enunciado.
- Verificação da suficiência dos testes e análise dos resultados do `evidencias.log`.
- Orientação para organizar o relatório conforme o Guia de Relatórios.
- Ajuda para resolver os conflitos de commits e sincronizar o repositório com o GitHub.

**O que validei manualmente e como:** A implementação das funções foi feita por mim, e os testes foram executados localmente no WSL. Compilei os arquivos utilizando o Makefile, executei o runner fornecido pelo professor e acompanhei as mensagens apresentadas no terminal. Também verifiquei o código de saída do programa e repeti os testes 100 vezes em duas sessões, obtendo sucesso nas execuções registradas. Os comandos e resultados podem ser conferidos no arquivo evidencias.log.

# 8. Reflexão Final

**Reflexão:** Apesar de esta implementação ter sido menor que as anteriores, precisei estudar melhor o funcionamento dos locks e da exclusão mútua. Minha maior dificuldade foi entender como sincronizar os passageiros e o vagão sem que as threads interferissem umas nas outras. Para isso, utilizei três contadores, um mutex e duas variáveis de condição. Também preferi separar os arquivos para organizar melhor o código, já que nas atividades anteriores eles acabaram ficando muito longos. Os testes apresentaram resultados positivos, mas percebi que poderia ter planejado mais situações específicas desde o começo, principalmente por se tratar de um problema de concorrência.

# 9. Checklist Final de Entrega

| Item | Confirmado? |
|---|---|
| Compilei do zero seguindo só o que está no relatório | Sim |
| Rodei os testes e evidencias.log foi gerado | Sim |
| Tenho prints obrigatórios | Sim |
| Testei pelo menos um caso limite e um caso inválido | Parcial |
| Preenchi a seção de uso de IA | Sim |
| Revisei o relatório e removi frases genéricas / vazias | Sim |

# 10. Se eu tivesse mais 2 horas

Com mais tempo, eu faria testes adicionais para verificar situações de concorrência que talvez não tenham aparecido no runner, principalmente quando vários passageiros tentam embarcar ao mesmo tempo. Também testaria entradas inválidas, como uma quantidade negativa de assentos, para entender como o programa se comportaria nesses casos. Por fim, organizaria melhor os registros do terminal e revisaria os nomes dos contadores, deixando mais clara a diferença entre reservar um assento e confirmar o embarque.